
Machine Learning

Mathematics and Computing

TUTORIAL 4

Students:

Eloi Rodríguez Rodríguez

ID: 100496948

100496948@alumnos.uc3m.es

Leyre Ruiz de Alegría Bravo

ID: 100497014

100497014@alumnos.uc3m.es

Exercise 1: Basic Implementation and Execution Modes

1. Open `qtable.txt` with a text editor. Explain what the rows and columns represent.

Each row of the Q-table corresponds to one of the 16 states in the 4×4 FrozenLake grid, numbered from 0 to 15 as shown below:

0	1	2	3
4	5 (H)	6	7 (H)
8	9	10	11 (H)
12 (H)	13	14	15 (G)

Each column represents one of the four possible actions: Left (0), Down (1), Right (2), and Up (3). The entry $Q(s, a)$ stores the expected cumulative reward of taking action a from state s . A higher value indicates a more promising action.

Several patterns are visible in the trained table. The holes (5, 7, 11, 12) and the goal (15) contain only zeros, since the episode ends upon reaching them and no further actions are taken. Conversely, state 14 has a Q-value of 1.0 for the action Right, because moving right from that cell leads directly to the goal. Values in the remaining cells decrease as they get farther from the goal, reflecting the cumulative effect of the discount factor $\gamma = 0.95$ applied at each step (e.g. 0.95, 0.9025, 0.8574, ...).

State	Left (0)	Down (1)	Right (2)	Up (3)
0	0.7351	0.7738	0.7738	0.7351
1	0.7351	0.0000	0.8145	0.7696
2	0.7685	0.8574	0.7736	0.8144
3	0.8145	0.0000	0.6183	0.7420
4	0.7738	0.8145	0.0000	0.7351
5	0.0000	0.0000	0.0000	0.0000
6	0.0000	0.9025	0.0000	0.8145
7	0.0000	0.0000	0.0000	0.0000
8	0.8145	0.0000	0.8574	0.7738
9	0.8145	0.9025	0.9025	0.0000
10	0.8574	0.9500	0.0000	0.8574
11	0.0000	0.0000	0.0000	0.0000
12	0.0000	0.0000	0.0000	0.0000
13	0.0000	0.9025	0.9500	0.8573
14	0.9025	0.9500	1.0000	0.9025
15	0.0000	0.0000	0.0000	0.0000

2. Identify the row corresponding to the state immediately to the left of the Goal. Which action has the highest value? Does this value make sense mathematically given the rewards?

State 14 is located immediately to the left of the goal (state 15).

Left (0)	Down (1)	Right (2)	Up (3)
0.9025	0.9500	1.0000	0.9025

The highest value belongs to action Right (action 2) with $Q(14, \text{Right}) = 1.0$. This is consistent with the reward structure, moving right from state 14 reaches the goal directly, yielding a reward of 1.0 with no further discounting. Applying the converged form of the Bellman equation gives

$$Q(14, \text{Right}) \longrightarrow R + \gamma \cdot \max_{a'} Q(15, a') = 1.0 + 0.95 \cdot 0 = 1.0.$$

The other actions from state 14 have lower values because they require at least one additional step to reach the goal. For instance, $Q(14, \text{Down}) = 0.95$ corresponds to reaching the goal in two steps: the immediate reward is 0, and the future reward is discounted once ($1.0 \times 0.95 = 0.95$). Each additional step multiplies the value by another factor of γ , producing the sequence 1.0, 0.95, 0.9025, 0.8574, ... observed throughout the table.

3. If you delete `qtable.txt` and immediately run the script in "PLAY" mode, what behavior would you expect from the agent on the screen? Why?

If `qtable.txt` is deleted and the script is run immediately in PLAY mode, the Q-table is initialised to all zeros. During play the agent selects actions greedily via $\arg \max_a Q(s, a)$. When every Q-value in a row is zero, NumPy's `argmax` returns the first index, which is action 0 (Left).

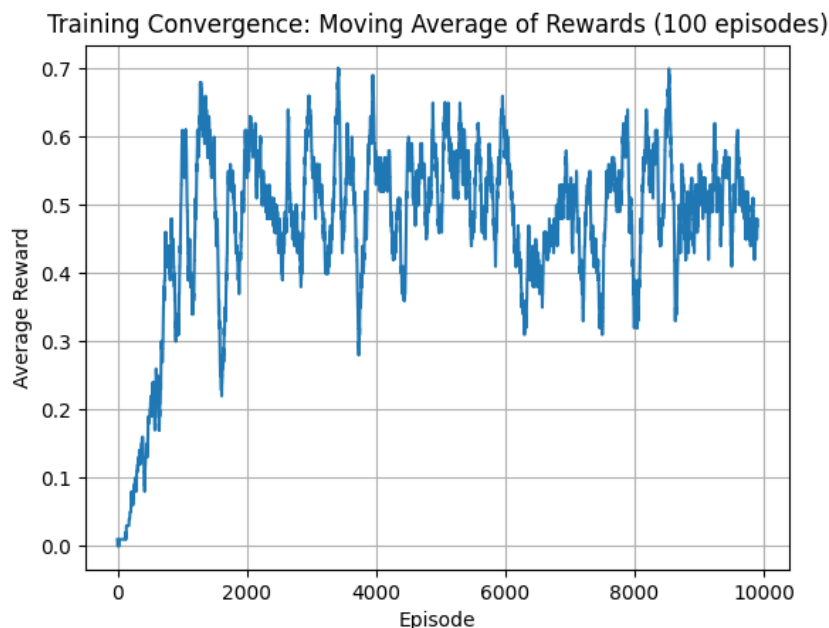
As a result, the agent will repeatedly attempt to move left regardless of its position. In most cases it will remain stuck against the left wall of the grid and never reach the goal, since it has no learned policy to guide it. This demonstrates that without prior training the greedy strategy is completely uninformed and effectively useless.

Exercise 2: Stochastic Environments

1. What is going on with the agent's movements when `is_slippery=True`? Does the training converge as fast as in the deterministic case?

When `is_slippery=True`, the FrozenLake environment implements a stochastic transition model, i.e., the agent's chosen action is only executed with probability $\frac{1}{3}$. Therefore, with the remaining probability of $\frac{2}{3}$, the agent slips to one of the two perpendicular directions instead. For example, choosing Down may result in moving Left or Right with equal probability $\frac{1}{3}$ each. This means the agent loses direct control over its trajectory and must learn a policy that is robust to unintended movements.

As shown in Figure , the training curve is substantially noisier and slower to stabilise compared to the deterministic case. In the deterministic environment, the moving average of rewards rises steeply and converges near 1.0 within a few thousand episodes. Now, with the stochasticity introduced, the average reward climbs to roughly 0.5–0.7 but oscillates with high variance throughout the entire 10 000-episode run, never settling. The additional noise arises because the same action from the same state can produce very different outcomes across episodes, which injects variance into every Q-update and makes it harder for the algorithm to associate actions with their true expected returns.



2. Look at the maximum average reward your agent achieves at the end of the stochastic training. Does it reach a solid 1.0 (100% success rate)? Explain why this happens given the nature of the slippery ice.

The maximum average reward observed is approximately 0.70, and the value at the end of training hovers around 0.50–0.55. So it does not reach 1.0.

This is a fundamental consequence of the stochastic transition model. Even if the agent has learned a theoretically optimal policy, it cannot guarantee reaching the goal on every episode because the environment may slip it into a hole against its will. More precisely, there exist states where any action carries a non-zero probability of transitioning into an adjacent hole. The resulting success rate is therefore strictly less than 1.0, reflecting the uncertainty of the environment rather than any deficiency in the learning algorithm.

3. Analyze the new Q-table. Compare the Q-values of a safe cell that is immediately adjacent to a hole in both scenarios (deterministic vs. stochastic). Why might the optimal action in the stochastic scenario NOT be the shortest path to the goal?

We compare state 6 (column 2, row 1), which lies between hole 5 on its left and hole 7 on its right. The Q-values in both scenarios are:

Scenario	Left (0)	Down (1)	Right (2)	Up (3)
Deterministic	0.0000	0.9025	0.0000	0.8145
Stochastic	0.0102	0.0003	0.0001	0.0108

In the

deterministic case. , left leads deterministically into hole 5 ($Q = 0$) and Right into hole 7 ($Q = 0$). Down is the direct path to state 10 and then to the goal, so $Q(6, \text{Down}) = 0.9025$. The greedy policy correctly chooses the shortest safe path.

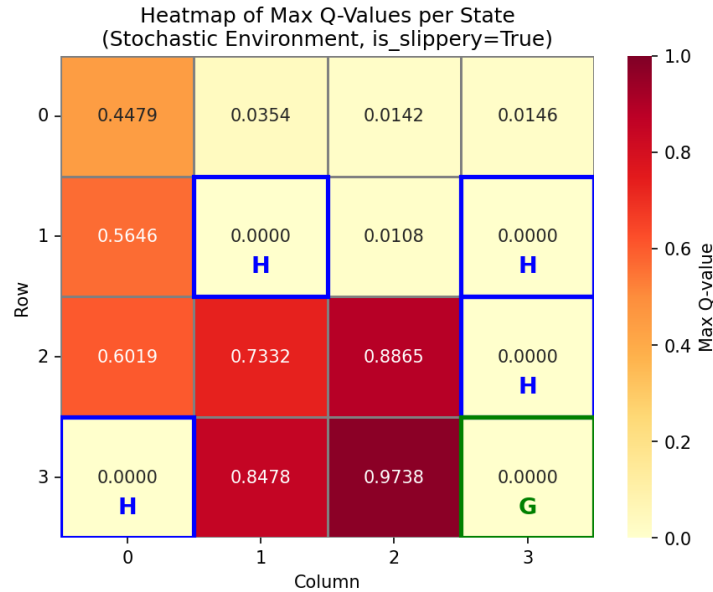
On the other hand, in the

stochastic case. , down is nominally still the shortest path, but in the slippery environment issuing a Down command from state 6 has a $\frac{1}{3}$ chance of slipping Left (into hole 5) or Right (into hole 7). The expected reward of trying to go Down is therefore severely penalised by the high probability of falling into an adjacent hole. The agent instead learns to prefer Up ($Q = 0.0108$), retreating away from the goal to reach the safer corridor at the top of the grid. Although this route is longer, it reduces the risk of a lethal slip. This illustrates a key principle of stochastic RL: the optimal policy maximises expected return, not path length, and may deliberately take a detour to avoid high-risk shortcuts.

Overall, Q-values in the stochastic table are dramatically lower than in the deterministic one because even the best achievable expected reward under uncertainty is much smaller than the certain reward in the deterministic setting.

4. Generate a heatmap of the maximum Q-values for each state from your final Q-table. Some zones of the map will have values equal to 0.0. Is this because those states are dangerous, or because the agent never explored them? Justify your answer.

Figure shows the heatmap of $\max_a Q(s, a)$ for each state in the stochastic Q-table.



Two distinct reasons explain the zero-valued states:

1. **Terminal states (holes and goal).** States 5, 7, 11, and 12 are holes, and state 15 is the goal. All five display $\max_a Q(s, a) = 0$ because the episode ends the instant the agent enters them. No action is ever taken from these states, so their Q-values remain at the initialisation value of zero throughout training. These zeros reflect the danger (holes) or termination (goal) nature of the state, not a lack of exploration.
2. **Rarely rewarding states (upper-right region).** States 1, 2, 3 (top row) and state 6 have Q-values that are effectively near zero (0.0108–0.0354). These states were visited by the agent during exploration, but under the slippery dynamics they lie on paths that rarely lead to the goal, either because they are surrounded by holes or because the policy converged to a safer route through the left column. Their low values confirm that they were explored, but they simply offer little expected reward under the stochastic model.

In contrast, the left-column and bottom-row states (0, 4, 8, 9, 10, 13, 14) show progressively higher values as they approach the goal, confirming that the agent successfully learned a viable policy along this safer corridor.

Conclusions

This tutorial highlighted how the introduction of stochasticity fundamentally changes the behaviour of a Q-learning agent. In the deterministic environment the agent converges quickly to a nearly perfect policy, as every action reliably produces the intended transition. In the stochastic case the inserted randomness of the slippery ice forces the agent to trade path length for risk of falling, leading to longer but safer routes, substantially lower Q-values, slower and noisier convergence, and a success rate capped well below 100%.

These results illustrate two core concepts of reinforcement learning in stochastic MDPs: The optimal policy maximises expected return rather than shortest distance, and the theoretical performance ceiling is set by the environment’s irreducible uncertainty, not by the algorithm’s ability to learn.